A Multi-Task Spatiotemporal Graph Neural Network Framework for Threat Detection, Propagation Forecasting, and Explainable Alert Generation

Architecture, Formal Properties, and a Feasibility Study on UNSW-NB15

Margarita Baimel Lerman, TalTech – Tallinn University of Technology, baimelrita@gmail.com
Eduard Petlenkov, TalTech – Tallinn University of Technology, eduard.petlenkov@taltech.ee
Ilan Aharon, Ariel University, ilanah@ariel.ac.il
Moshe Sitbon, Ariel University, moshesi@ariel.ac.il
Leena Santosh, Pandit Deendayal Energy University, leena80santosh@gmail.com
Dmitry Baimel, SCE – Shamoon College of Engineering, dmitrba@sce.ac.il

Abstract

This paper presents a *framework* – an architecture, its formalisation, and a feasibility study – for multi-task threat detection over dynamic network graphs, posing detection, blast-radius forecasting, and sparse explanation as joint objectives of one spatiotemporal graph neural network (ST-GNN) rather than independent modules. Traffic is a dynamic directed threat graph whose nodes are IP addresses and whose edges are flows aggregated into five-minute meta-edges. Three heads share a GAT-based encoder with EMA node memory: a Detection Head classifying window content, a Blast-Radius Head estimating node compromise probability, and an Explainability Head extracting minimal MITRE-aligned causal subgraphs. Contributions: (i) a complete formalisation, including blast-radius forecasting as a supervised problem with explicit input/output spaces, objective, and assumptions; (ii) a complexity analysis of every pipeline stage and head; (iii) a methodological result on label aggregation – assigning a window its highest-index attack class, rather than its most frequent one, renders co-occurring classes structurally undetectable regardless of model capacity – with two rare-class-recovery mechanisms, a multi-label Detection Head and a per-edge classification head; and (iv) a proposed evaluation protocol specifying experiments required to validate the framework. A feasibility study on the raw, IP-address-inclusive UNSW-NB15 export shows stable training across five seeds and previously unrecoverable classes (F1 = 0) reaching edge-level F1 ≥ 0.94, with edge-level Macro F1 = 0.978 ± 0.018 and window-level Macro F1 = 0.849 ± 0.066. Comparative baselines, ablation, and Blast-Radius Head evaluation are deliberately outside this paper's scope.

Keywords: spatiotemporal graph neural network; threat detection and response; multi-task learning; dynamic threat graph; explainable AI; MITRE ATT&CK; label aggregation; rare-class recovery; multi-label graph classification; intrusion detection; UNSW-NB15

1 Introduction

1.1 Background and Problem Statement

Security Operations Centers (SOCs) confront an asymmetric threat landscape: adversaries execute multi-stage intrusion campaigns that traverse dozens of hosts, exploit relational dependencies between users, services, and processes, and maintain dwell times measured in months – while defenders rely on systems that process security events in isolation [2]. This structural mismatch between attacks that are inherently graph-shaped and detectors that are inherently event-centric underlies a persistent gap between machine-learning benchmark performance and real-world SOC deployment.

Three interdependent deficiencies characterise the current state of the art.

Operational value gap. Prevailing detection systems are evaluated on classification metrics – accuracy, ROC-AUC, F1-score – that are decoupled from the operational outcomes SOC practitioners are accountable for: alert fatigue, time-to-decision, blast-radius reduction, and containment confidence [18]. A system that achieves a state-of-the-art F1 but cannot answer *what should the analyst do next?* delivers limited actionable utility in production environments.

Propagation blindness. Existing ML detectors resolve the classification question – *is this event anomalous?* – without addressing the response question: *which assets will be compromised next, and how rapidly?* In lateral movement campaigns, blast-radius forecasting is operationally comparable in urgency to initial detection; acting without propagation awareness yields incomplete containment and unnecessary escalation of incident scope.

Non-causal explanations. Post-hoc attribution methods supply feature importance scores but do not reconstruct the causal trajectory of an attack. The practical consequence is well-documented algorithm aversion: analysts default to rule-based investigation workflows even when ML classifiers are statistically superior [15], reducing the operational return on the detection infrastructure as a whole.

These deficiencies share a structural origin: event-centric representations discard the relational, temporal, and causal context that defines multi-stage intrusion campaigns. Spatiotemporal graph neural networks (ST-GNNs) are architecturally suited to address all three within a unified formulation – modelling network entities as nodes in a dynamic graph, propagating representations along edges that reflect real communication and dependency paths, and supporting joint optimisation of detection, propagation forecasting, and explanation objectives within a single end-to-end training procedure.

1.2 Scope and Contribution Type of This Paper

This paper is a *framework paper*. Its claim is that the proposed architecture is a coherent, well-specified, and computationally tractable way to pose detection, propagation forecasting, and explanation as one learning problem – not that it outperforms alternative architectures. Accordingly, the paper delivers a full architectural specification, a formalisation of each learning problem the architecture solves, a complexity analysis, a methodological result on label aggregation, and a feasibility study demonstrating that the framework can be instantiated end-to-end on real, IP-address-labelled traffic and trained to a stable operating point.

What this paper deliberately does **not** do is equally explicit. It reports no comparative baselines, no ablation study, and no evaluation of the Blast-Radius Head. A proposed evaluation protocol (Section 6) specifies exactly which experiments would be required to substantiate the framework's performance claims; those experiments are the subject of a separate, dedicated evaluation study. Every capability that has not been measured is described in conditional or future tense throughout, and no unmeasured capability appears in the list of contributions or in the abstract.

1.3 Research Focus

The framework addresses multi-stage network intrusion campaigns, with lateral movement and identity-driven attack progressions as the primary target domain [11]. This scenario is graph-structured by construction – spanning hosts, services, and network flows – blast-radius directly determines containment scope, and MITRE ATT&CK provides a structured ontology against which tactic-level explanations can be grounded and evaluated [19].

The feasibility study is conducted on UNSW-NB15 [25], a publicly accessible, IP-address-labelled network intrusion dataset produced by the Australian Centre for Cyber Security. It is important to state plainly what this dataset is: a synthetic testbed containing 48 unique IP addresses, with real attack traffic mixed into generated background traffic. It is an appropriate vehicle for demonstrating that the pipeline runs end-to-end and produces a learnable signal; it is not evidence about behaviour on enterprise-scale telemetry, and no claim of that kind is made anywhere in this paper.

1.4 Research Objective

Design, formalise, and demonstrate the feasibility of a Spatiotemporal Graph Neural Network architecture for Threat Detection and Response (TDR) that: (1) performs causal threat detection over dynamic threat graphs; (2) specifies blast-radius forecasting – which nodes are at highest risk of imminent compromise – as a jointly trained supervised objective; and (3) produces sparse, MITRE-aligned causal explanations via an Explainability Head that annotates minimal attack subgraphs with tactics and techniques [8].

1.5 Research Questions

Overarching research goal. How can an ST-GNN-based threat detection system be designed so that accurate detection, blast-radius forecasting, and causally grounded explanations are produced by a single model aligned with real SOC workflows?

RQ1 – Multi-stage threat detection. How can ST-GNNs over dynamic threat graphs be formulated to detect complex, multi-stage attack campaigns that event-centric or black-box models represent poorly [24]?

RQ2 – Blast-radius forecasting. How can threat propagation over a dynamic graph be *posed as a learning problem* – with a well-defined input space, label-generating process, objective, and assumptions – such that future at-risk nodes and likely attack paths can be forecast, integrating data-driven patterns with MITRE ATT&CK transition knowledge?

RQ3 – Sparse causal explainability. How can a sparse causal explainability mechanism be embedded as a trained head, rather than a post-hoc module, so that explanations refer to the same representation that produced the decision?

This paper answers RQ1 and RQ3 at the level of architecture, formalisation, and feasibility, and answers RQ2 at the level of formalisation only; the empirical components of all three are addressed by the evaluation protocol in Section 6.

1.6 Contributions

1. **A multi-task ST-GNN architecture**, specified in full, that jointly optimises detection, blast-radius forecasting, and sparse MITRE-aligned explanation over dynamic IP-level network graphs with a shared encoder and conditionally instantiated heads.
2. **A formalisation of blast-radius forecasting as a supervised learning problem** (Section 4.1): input space, output space, label-generating process distinguishing first compromise from persistence, objective functional, and the five assumptions the formulation requires – including the assumption whose failure makes UNSW-NB15 unsuitable for evaluating this head.
3. **A methodological result on label aggregation** (Section 4.2): a formal statement and proof that under single-label window aggregation, any attack class that never attains plurality in a window is unlearnable and unmeasurable regardless of model capacity, together with the identification of the max-index aggregation rule as a distinct and more severe error; plus two rare-class-recovery mechanisms – a multi-label Detection Head variant and a per-edge classification head – that remove this constraint.
4. **A complexity analysis** of every pipeline stage and of each head (Section 4.3), establishing that the framework's cost is linear in the number of meta-edges per snapshot and identifying the node-memory recurrence as the sole barrier to snapshot-level parallelism.
5. **A proposed evaluation protocol** (Section 6) specifying the baselines, ablations, and falsification criteria required to validate the framework, and a feasibility study (Section 5) establishing that the framework trains stably on real IP-labelled traffic.

1.7 Organisation

Section 2 reviews related work. Section 3 specifies the framework: design rationale, pre-processing, graph definition, features, encoder, heads, and composite objective. Section 4 develops the framework's formal properties: the blast-radius learning problem, the label-aggregation result, and the complexity analysis. Section 5 reports the feasibility study. Section 6 proposes the evaluation protocol required to validate the framework. Section 7 states limitations, Section 8 outlines future research, and Section 9 concludes.

2 Related Work

2.1 Machine Learning for Intrusion Detection and the Operational Gap

Deep learning approaches to intrusion detection have been surveyed extensively [2], and the mismatch between benchmark performance and operational usefulness has been documented as a structural rather

than incidental problem [18]. Class imbalance is a recurring obstacle in this literature, addressed through resampling, cost-sensitive weighting, and loss reshaping such as focal loss [5, 9]. A point largely absent from this line of work, and central to Section 4.2 below, is that when flow-level telemetry is aggregated into time windows, the *label aggregation rule itself* – not only the class distribution – determines which categories remain learnable.

2.2 Graph and Dynamic-Graph Learning for Network Security

Graph neural networks provide a natural representation for traffic and identity relationships. Foundational architectures include graph convolutional networks [7], inductive neighbourhood sampling [3], and attention-based aggregation [20], surveyed comprehensively in [22]. Dynamic extensions adapt these to evolving topologies, either by evolving the network parameters over time [13] or by learning dynamic graph representations specifically for attack detection [1]. Spatiotemporal formulations have been applied to advanced persistent threat detection [8], and unified graph learning frameworks have been proposed that couple detection with explanation [24]. On the UNSW-NB15 dataset family, self-supervised edge-embedding approaches [26] and timestamp-aware spatiotemporal contrastive formulations [27] represent the closest published work; both address a differently posed task – principally binary or self-supervised detection over pre-extracted NetFlow records – and are named here as the comparison targets for the evaluation protocol of Section 6 rather than as baselines evaluated in this paper.

2.3 Provenance, Kill Chains, and Multi-Stage Attribution

A parallel line of work reconstructs attack progression from provenance rather than learning it from traffic statistics: tactical provenance analysis for endpoint detection and response [4], real-time APT detection by correlating suspicious information flows against a kill-chain model [11], and knowledge-graph-driven kill-chain modelling for multi-stage attribution [17]. These systems establish that multi-stage campaigns are best reasoned about relationally, and motivate the MITRE-aware attention mechanism described in Section 3.6, but they rely on curated provenance graphs rather than learned representations of raw traffic.

2.4 Propagation and Blast-Radius Modelling

Graph-based threat propagation prediction has been studied as a distinct task from detection [12]. The framework proposed here differs in treating propagation as a head of the same model that performs detection, so that the propagation forecast is conditioned on the representation that produced the detection decision. As Section 4.1 makes explicit, this formulation imposes a requirement on the data – observable first-compromise events – that most public intrusion benchmarks, including UNSW-NB15, do not satisfy.

2.5 Explainability for Graph Models in Security

Explainability methods for GNNs have been organised into a taxonomy of instance- and model-level approaches [23], and explainable AI in cybersecurity specifically has been reviewed in [15], where analyst trust is identified as the binding operational constraint. SHAP-guided graph learning has been proposed

as an explainable IDS framework [21]. The distinction relevant here is between post-hoc attribution, which explains a surrogate of the decision, and a jointly trained explanation head, which is constrained during training to compete for the same gradient as the detection objective.

2.6 Multi-Task Learning, Anomaly Detection, and Supporting Components

Multi-task learning in deep networks, including the risks of negative transfer and the role of loss weighting, is surveyed in [14]; these considerations directly inform the design rationale in Section 3.1. Unsupervised outlier node detection on attributed graphs has been benchmarked systematically [10], and provides the literature basis for the complementary anomaly-detection layer discussed in Section 3.9 – a layer that is described here as a planned extension rather than an evaluated component. Time2Vec [6] supplies the learnable temporal encoding used for edge features, MITRE ATT&CK [19] supplies the tactic ontology used for semantic grounding, and UNSW-NB15 [25] and CSE-CIC-IDS2018 [16] are the benchmark datasets referenced in this work.

3 Proposed Architecture

The architecture is presented here as originally designed. Its components are unchanged from the model that produced the results in Section 5; what has changed relative to earlier drafts of this work is which of its outputs are reported as measured, and which are reported as implemented-but-not-yet-evaluated. That distinction is made explicit at the point in this section where it first becomes relevant (Blast-Radius Head, §3.7; anomaly-detection layer, §3.9), and is carried through consistently into Sections 4–7.

3.1 Pre-processing: Hierarchical Entity Aggregation

To address the "graph explosion" inherent in raw telemetry, we introduce a Temporal Entity Aggregation Layer. Instead of mapping every log entry to a unique node, we collapse topologically equivalent events into Meta-Nodes.

Let $\mathscr{E} = \{e_1, \ldots, e_n\}$ be the stream of raw events. We define a mapping function $\Phi: \mathscr{E} \to V_{meta}$ that groups events based on the tuple:

$$\Phi(e_i) = \left(\mathrm{HostID}(e_i),\ \mathrm{UserID}(e_i),\ \lfloor t_i / \Delta\tau \rfloor\right) \tag{1}$$

where $\Delta\tau$ is a short aggregation window (e.g., 5 minutes). A Meta-Node $v_k \in V_{meta}$ aggregates features from all raw events using a permutation-invariant pooling function:

$$h_{v_k} = \mathrm{MLP}_{\mathrm{agg}}\left(\bigoplus_{e_i \in S_k} \mathrm{feat}(e_i) \right) \tag{2}$$

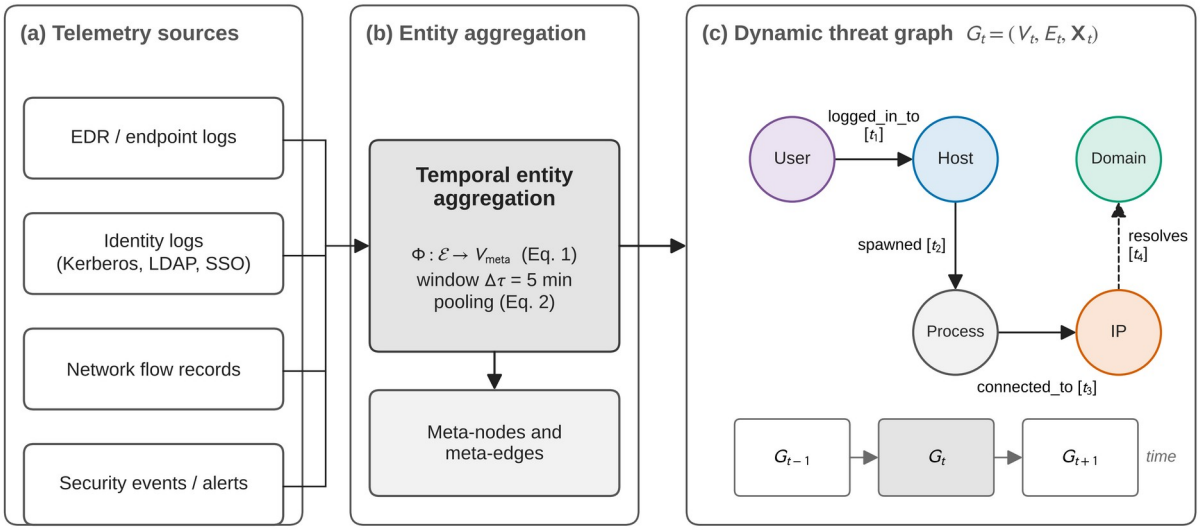
This reduces the graph size by orders of magnitude while preserving the causal structure required for lateral-movement detection.

3.2 Formal Graph Definition

At each time step t, we represent the enterprise environment as a dynamic directed threat graph [1, 4]:

$$\mathcal{G}_t = (V_t, E_t, \mathbf{X}_t) \tag{3}$$

where V_t is the set of heterogeneous nodes (Users, Hosts, Processes, IPs, Domains) and E_t represents observed interactions (e.g., Remote-Logins, Process-Injections, Network-Connections). The feature tensor $\mathbf{X}_t$ encapsulates both static attributes and dynamic structural properties within a sliding time window (Fig. 1).



Node and edge types shown are those supported by the framework; the feasibility study (Section 5) instantiates IP nodes and flow meta-edges only.

Fig. 1. *Formal Graph Definition Architecture.*

3.3 Scalable Graph Metrics as First-Class Features

To bridge the gap between raw telemetry and topological importance, graph metrics are integrated directly into the node feature vectors. Recognising that global metrics are computationally prohibitive for real-time inference on massive graphs, we use Approximate Localized Metrics updated incrementally (Fig. 2).

3.3.1 Approximate Eigenvector Centrality. Centrality is treated as a learnable embedding updated via power iteration within the GNN's message-passing layers, highlighting operationally critical nodes such as jump servers or domain controllers without global graph traversals:

$$c_i^{(t)} \leftarrow \frac{1}{\lambda} \sum_{j \in \mathcal{N}(i)} A_{ij}\, c_j^{(t-1)} \tag{4}$$

3.3.2 Temporal Structural Deviations (Burstiness). To detect sudden spikes in a host becoming a lateral-movement hub, we compute localized burstiness by comparing current degree activity to an exponential moving average (EMA):

$$\delta_{\text{burst}}(i) = \frac{\deg_i(t) - \text{EMA}(\deg_i, t-1)}{\sigma_{\deg_i} + \varepsilon}$$

(5)

3.3.3 Local Connectivity Metrics. Standardised features including in/out-degree and weighted interaction frequency.

Edge representation. For each edge $e_{ij} \in E_t$, we define a feature vector $\mathbf{x}_{ij}$ as:

$$\mathbf{x}_{ij} = \left[\varphi_{\text{struct}}(i,j) \,\|\, T2V(t) \,\|\, \mathbf{e}_{\text{mitre}}(t)\right]$$

(6)

where $\varphi_{\text{struct}}(i,j)$ captures structural relations (e.g., shared privilege boundaries or host membership); T2V(t) is a Time2Vec temporal encoding [6] capturing periodicity and relative timing of interactions; and $\mathbf{e}_{\text{mitre}}(t)$ is an optional embedding of the MITRE ATT&CK techniques most likely associated with the interaction, enabling the model to align structural anomalies with known adversarial behaviours.

3.3.4 Community Representation. Handled through the Temporal Entity Aggregation layer, which collapses topologically equivalent events into Meta-Nodes using the mapping function

$$\Phi(e_i) = \left(\text{HostID}(e_i),\ \text{UserID}(e_i),\ \lfloor t_i / \Delta\tau \rfloor\right)$$

(7)

ensuring the model reasons over functional groups rather than fragmented logs.

3.3.5 Distance to Crown Jewels. Modelled by calculating the shortest path from a node to the set of critical assets C (e.g., domain controllers identified via centrality):

$$D_{CJ}(i) = \min_{j \in C} \text{dist}(i,j)$$

(8)

This provides a spatial priority signal for the Blast-Radius Head.

3.3.6 Causal Consistency. Enforced via the loss term L_{causal}, which penalises explanation edges that do not align with the known MITRE transition matrix:

$$L_{\text{causal}} = \sum_{(i,j) \in E_{\text{exp}}} \text{dist}\left(\text{state}(i,j),\, T_{\text{mitre}}\right)$$

(9)

where T_{mitre} represents valid adversarial progressions.

Key insight. These metrics enable the ST-GNN to prioritise high-impact regions of the graph, focusing both detection and forecasting on nodes that serve as potential bottlenecks or pivots for a multi-stage campaign.

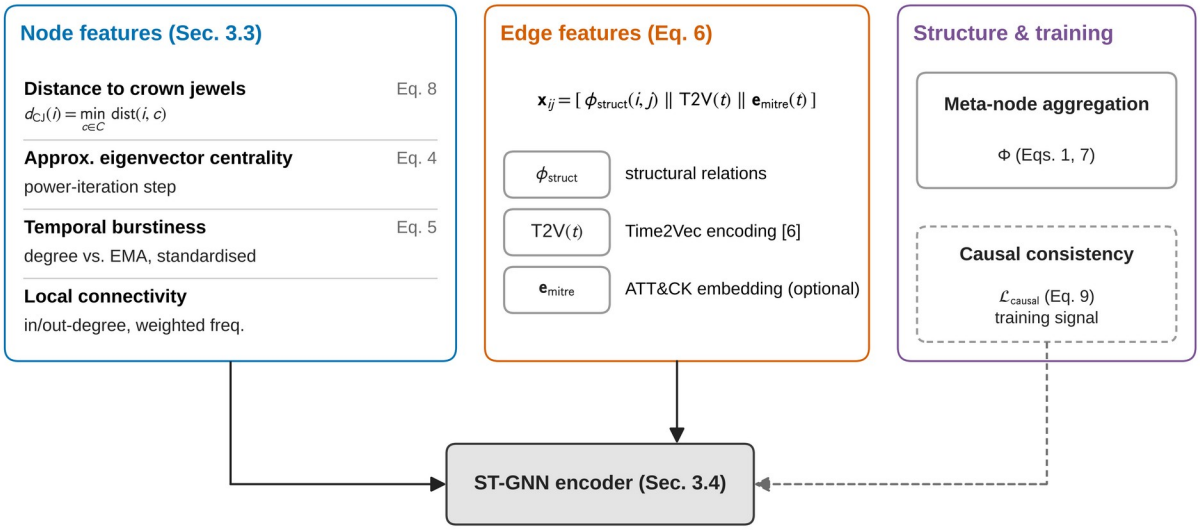


Fig. 2. *Feature Engineering Pipeline for Dynamic Threat Graphs.*

3.4 Causal-Temporal ST-GNN Engine with MITRE-Aware Threat Flows

A spatiotemporal GNN serves as the causal reasoning engine: it encodes the dynamic threat graph, aggregating information over space (graph structure) and time (event sequences), and uses graph-temporal attention to identify which nodes/edges and time steps most strongly contribute to suspicious patterns versus benign background noise.

3.4.1 Spatiotemporal Message Passing. Let $h_i^{(l)}$ denote the hidden representation of node i at layer l, initialised from its raw features and graph metrics at time t. A single spatiotemporal GNN layer updates node embeddings via attention-weighted aggregation over neighbours [20]:

$$h_i^{(l+1)} = \sigma\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(l)} \, W^{(l)} h_j^{(l)} \right)$$

(10)

where $\mathcal{N}(i)$ is the set of neighbours of node i, $W^{(l)}$ is a learnable weight matrix at layer l, $\alpha_{ij}^{(l)}$ is an attention coefficient quantifying the importance of neighbour j for updating i, and $\sigma(\cdot)$ is a non-linear activation (e.g., ReLU).

The attention coefficient combines structural, temporal, and MITRE-based context:

$$\alpha_{ij}^{(l)} = \text{softmax}_{j \in \mathcal{N}(i)}\left(a^\top \left[W^{(l)} h_i^{(l)} \,\|\, W^{(l)} h_j^{(l)} \,\|\, e_{ij}^t \right] \right)$$

(11)

where a is a learnable vector, ‖ denotes concatenation, and e_{ij}^{t} is the edge feature vector from Eq. (6). This formulation ensures that edges consistent with known attack patterns (via φ_{mitre}) and temporally salient interactions receive higher attention, forming the basis for both detection and explanation [22].

3.5 Domain Knowledge Injection via MITRE ATT&CK and Semantic Mapping

To move from static event assignment toward a representation of adversarial intent, the system integrates a Semantic Mapping mechanism based on ATTACK-BERT [19]. This aligns MITRE techniques with the dynamic threat graph across three dimensions.

Representing MITRE ATT&CK as a semantic auxiliary graph (Attack State Graph). Rather than a rigid technique graph, relationships are defined by semantic similarity and transition probabilities. Each MITRE technique is converted into a representative vector $E(v_i) \in \mathbb{R}^d$ using ATTACK-BERT, fine-tuned on cybersecurity-specific corpora. The relationship between techniques v_i and v_j (transitions between phases in the Cyber Kill Chain) is defined by cosine similarity:

$$\mathrm{sim}(v_i, v_j) = \frac{v_i \cdot v_j}{\|v_i\|\|v_j\|}$$

(12)

A directed edge is created only if the similarity exceeds a threshold (typically 0.7–0.9), so the state graph represents logical, credible adversarial progressions.

Associating enterprise graph entities with MITRE techniques. Instead of rule-based (keyword) mapping, the system uses BERT to semantically analyse event descriptions (e.g., payloads or system logs). When the Detection Head identifies a suspicious subgraph, it compares its representation to the BERT embeddings of MITRE techniques – Causal Detection – prioritising events semantically consistent with known attack patterns and thereby reducing false positives caused by statistical noise.

Joint reasoning and decision making. The ST-GNN is trained to reason simultaneously over the enterprise threat graph and the semantic MITRE graph.

Explainability process. The Explainability Head leverages semantic mapping to transform internal attention weights into a structured Attack Story, selecting edges with the highest semantic similarity to MITRE techniques and presenting the analyst with a narrative linking physical system entities to adversarial tactics.

Blast-radius forecasting. When predicting threat propagation, the Blast-Radius Head is designed to forecast not only the next target node but also the semantic techniques the attacker is likely to deploy next, constraining predicted paths to be consistent with logical Cyber Kill Chain progression (e.g., Installation → C2). As Section 3.7 states explicitly, this design capability is implemented but not evaluated in this paper.

Operational summary. The integration of ATTACK-BERT is intended to transform the system from a passive anomaly detector into a forecasting tool that understands the attack lifecycle, allowing SOC analysts to validate alerts against a structured, goal-oriented attack narrative (Fig. 3).

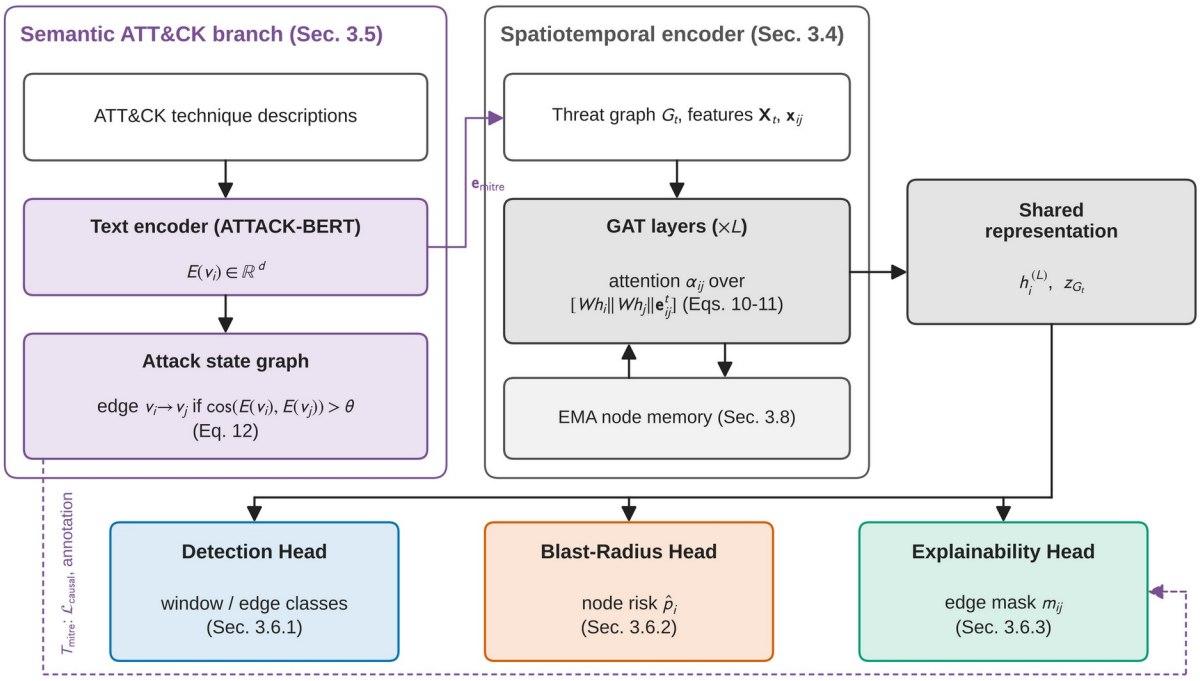


Fig. 3. *MITRE-Aware ST-GNN Engine for Causal Threat Reasoning.*

3.6 Triple Multi-Task Heads

3.6.1 Detection Head – Causal Threat Detection. Input: pooled node/edge embeddings over a time window (subgraph). Output: a label over detection classes (benign vs. malicious, or specific attack type/stage). Goal: detect active or emerging attack campaigns expressed as patterns in the graph – e.g., lateral-movement chains, coordinated login anomalies, privilege escalation (Fig. 4).

Given the final node embeddings $\{h_i^{(L)}\}$ ~i ∈ Vt~ at layer L, a subgraph-level embedding is computed using a permutation-invariant READOUT function (e.g., mean or attention pooling) [3]:

$$z_{G_t} = \mathrm{READOUT}\left(\{h_i^{(L)} : i \in V_t\}\right)$$

(13)

The detection logits are then given by

$$\hat{y}_{\mathrm{det}} = \mathrm{softmax}(W_{\mathrm{det}}\, z_{G_t} + b_{\mathrm{det}})$$

(14)

where W_{det} and b_{det} are learnable parameters and $\hat{y}_{det}$ is the predicted distribution over detection classes. The detection loss is standard cross-entropy:

$$L_{\mathrm{detect}} = \mathrm{CE}(y_{\mathrm{det}}, \hat{y}_{\mathrm{det}})$$

(15)

with y_{det} the ground-truth label for the current subgraph.

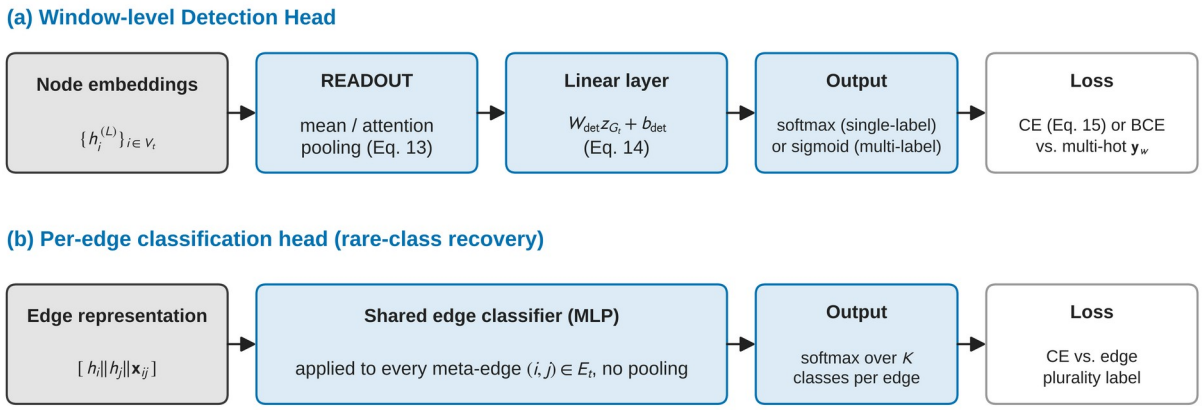


Fig. 4. *Pipeline of the Causal Detection Head.*

Rare-class recovery: multi-label detection and per-edge classification. Single-label, window-level classification has a structural blind spot on traffic where several attack types genuinely co-occur within the same aggregation window: whichever plurality label is assigned to a window necessarily discards every other attack type present in it, so a category that is common at the level of individual flows can still never win plurality in any window and remains permanently invisible to a single-label Detection Head, regardless of model capacity (this is formalised as Proposition 1 in Section 4.2). Two complementary, jointly trainable mechanisms address this.

Multi-label detection. The Detection Head is optionally trained as a multi-label classifier: each class receives an independent sigmoid output and binary cross-entropy loss, and a window's ground truth is a multi-hot vector marking every attack class whose share of that window's flows clears an explicit presence threshold (max(3, 1% of the window's flows), chosen to avoid re-introducing single-flow labelling noise), with the all-zero vector reserved for windows with no attack class present. This lets the head report every co-occurring attack type in a window instead of collapsing them to one winner.

Per-edge classification. A second, structurally simple head consumes the same per-edge representation already computed for the Explainability Head (concatenated source/destination node embeddings and assembled edge features) and predicts a class distribution for each individual meta-edge, trained with cross-entropy over edge-level plurality labels. Because meta-edges are not pooled across an entire window, this supervises every attack category in proportion to how many real flows of it exist in the data, independent of whether that category ever dominates a window.

Both heads are optional and add zero parameters or gradient to the shared encoder when disabled, following the same conditional-instantiation pattern used for the Blast-Radius and Explainability Heads.

3.6.2 Blast-Radius Forecasting Head – Threat Propagation Prediction. Instead of predicting whether an attack will occur, this head is designed to predict how the current threat is likely to spread [12] (Fig. 5).

For each node, estimate P(compromised_soon | current graph state). For each edge, estimate P(attacker traverses this edge next).

This is designed to yield: a **blast-radius map** – a ranked set of at-risk nodes (hosts, users, services) likely to be impacted if no containment action is taken; and probable **propagation paths**, constrained by both

the underlying network/identity topology and typical MITRE progression patterns (e.g., persistence → lateral movement → collection → exfiltration).

Node-level risk scoring. For each node $i \in V_t$, the Blast-Radius Head predicts a scalar risk score s_i based on its final embedding and the global subgraph embedding:

$$s_i = f_{\text{blast}}\left(h_i^{(L)}, z_{G_t}\right)$$

(16)

where f_{blast} is a small feed-forward network shared across nodes. Scores are converted into probabilities of near-term compromise via a sigmoid:

$$\hat{p}_i = \sigma(s_i)$$

(17)

with $\hat{p}_i = P(\text{compromised_soon} \mid G_t, i)$.

Blast-radius loss. Let $y_i^{\text{future}} \in \{0,1\}$ indicate whether node i becomes compromised within a future horizon $[t, t+\Delta]$ in the ground truth. The blast-radius forecasting loss is:

$$L_{\text{blast}} = \sum_{i \in V_t} \text{CE}\left(y_i^{\text{future}}, \hat{p}_i\right)$$

(18)

penalising discrepancies between predicted and actual future compromise for each node. This can be complemented in principle by ranking-based evaluation metrics such as Hits@K and NDCG to reflect prioritisation quality; Section 4.1 formalises the learning problem this loss is meant to solve, and Section 5.4 states explicitly why it is not evaluated in this paper.

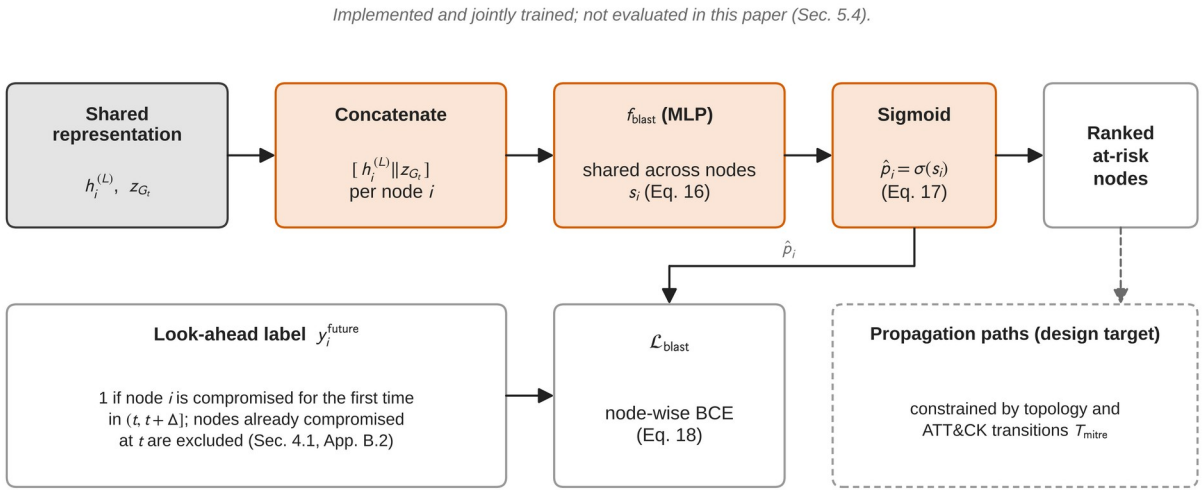


Fig. 5. *Blast-Radius Forecasting Head.*

3.6.3 Explainability Head – Sparse MITRE-Aligned Causal Subgraphs. The Explainability Head is not a post-hoc XAI module but an integral prediction head trained jointly with detection and forecasting [15]

(Fig. 6). It receives the ST-GNN's internal representations (node/edge embeddings, attention weights) and selects a minimal, high-impact causal subgraph – a small set of nodes and edges with the highest contribution to the detection decision and the blast-radius forecast, subject to a sparsity constraint (e.g., limiting the number of edges/nodes). It annotates this subgraph with relevant MITRE tactics/techniques, based on the mapping learned in Section 3.5.

The resulting explanation is designed to be: a **concise attack story** – which entities, in which order, under which techniques, led to the current threat state – and a **propagation preview** – which neighbouring entities are at elevated risk next, and via which techniques. This is intended to support SOC workflows by giving analysts a graph-based narrative they can validate, challenge, or act upon, rather than opaque scores.

Edge selection mask. The Explainability Head learns a soft mask $m_{ij} \in [0,1]$ for each edge $(i,j) \in E_t$, indicating how strongly this edge should be included in the explanation subgraph:

$$m_{ij} = \sigma\left(g(h_i^{(L)}, h_j^{(L)}, e_{ij}^t)\right)$$

(19)

where $g(\cdot)$ is a small neural scoring function and $\sigma(\cdot)$ is the sigmoid. The explanation subgraph is then defined as:

$$G_t^{\text{exp}} = (V_t, E_t^{\text{exp}}), \qquad E_t^{\text{exp}} = \{(i,j) \in E_t \mid m_{ij} > \tau\}$$

(20)

with $\tau \in (0,1)$ a threshold controlling the sparsity of the explanation.

Sparsity loss. To encourage minimal, analyst-usable explanations, a sparsity term is included:

$$L_{\text{sparsity}} = \lambda \sum_{(i,j)\in E_t} m_{ij}$$

(21)

where $\lambda > 0$ controls the trade-off between explanation compactness and fidelity. Lower L_{sparsity} corresponds to fewer edges being selected, and thus shorter and more focused attack stories [23].

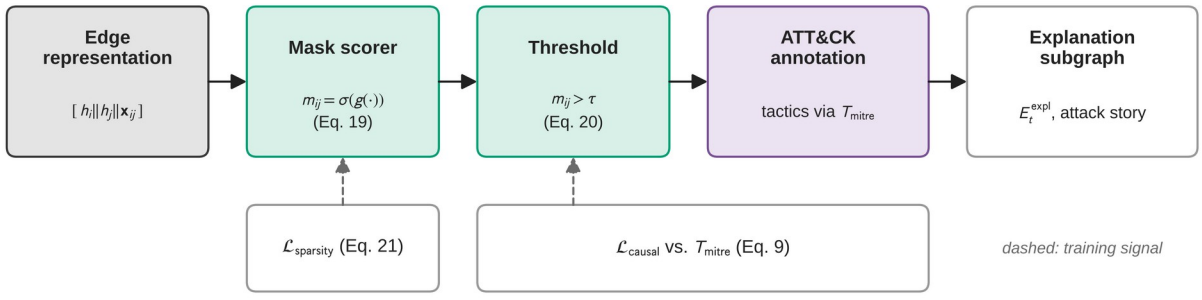


Fig. 6. *Explainability Head Architecture.*

3.7 Training Strategy Emphasising Causality and Sparsity

The model is trained with a multi-task composite loss:

$$L = L_{\text{detect}} + \alpha L_{\text{blast}} + \beta L_{\text{sparsity}} + \gamma L_{\text{causal}}$$

(22)

L_{detect} is the standard detection loss (Eq. 15) on benign/malicious labels or attack types. L_{blast} is the blast-radius forecasting loss (Eq. 18), penalising divergence between predicted at-risk node distributions and ground-truth compromised nodes in later time steps. $L_{sparsity}$ is the sparsity loss on the explanation subgraph (Eq. 21), encouraging minimal explanations by penalising large or diffuse subgraphs. L_{causal} is the causal-consistency loss (Eq. 9), encouraging alignment between explanation edges and known or plausible attack transitions, reducing reliance on spurious correlations.

Here, L_{detect} enforces accurate classification of malicious versus benign subgraphs, L_{blast} trains the model to predict future at-risk nodes, $L_{sparsity}$ encourages concise explanations, and L_{causal} aligns explanations with plausible MITRE-consistent attack progressions. The hyperparameters α, β, $\gamma > 0$ weight the relative importance of propagation accuracy, explanation compactness, and causal consistency respectively, allowing practitioners to tune the model to product requirements (e.g., favouring shorter explanations or more aggressive blast-radius prediction). All four terms are jointly trainable end-to-end; as Section 5 reports, the feasibility study trains the full composite objective, but reports measured results only for the terms whose ground truth is well-founded on the evaluation dataset (Section 4.1 and Section 5.4).

3.8 Node Memory

Node state is carried across sequential 5-minute snapshots via an exponential-moving-average (EMA) memory module (implemented in `trainer.py`, Section 3.10.3), which stores and updates IP-specific embeddings across time bins, allowing the model to retain host behaviour across the dataset's full duration rather than treating each snapshot independently. This mechanism is the basis for the complexity discussion in Section 4.3, where it is identified as the one component of the pipeline that is not embarrassingly parallel across snapshots.

3.9 Graph Anomaly Detection Integration (Planned Extension)

To complement the domain-aware ST-GNN, a graph-level anomaly detection layer (e.g., graph autoencoder or one-class models) trained over the learned embeddings would be able to identify rare or novel campaigns (zero-day patterns) that deviate from both normal behaviour and known MITRE patterns, and would surface statistical rarity scores (e.g., "top 0.5% most anomalous subgraphs") that could be combined with causal and MITRE explanations for a richer threat assessment (Fig. 7). This layer is architecturally compatible with the shared encoder described above but has not been implemented or evaluated as part of this work; it is discussed further as a future-research direction in Section 8.1.

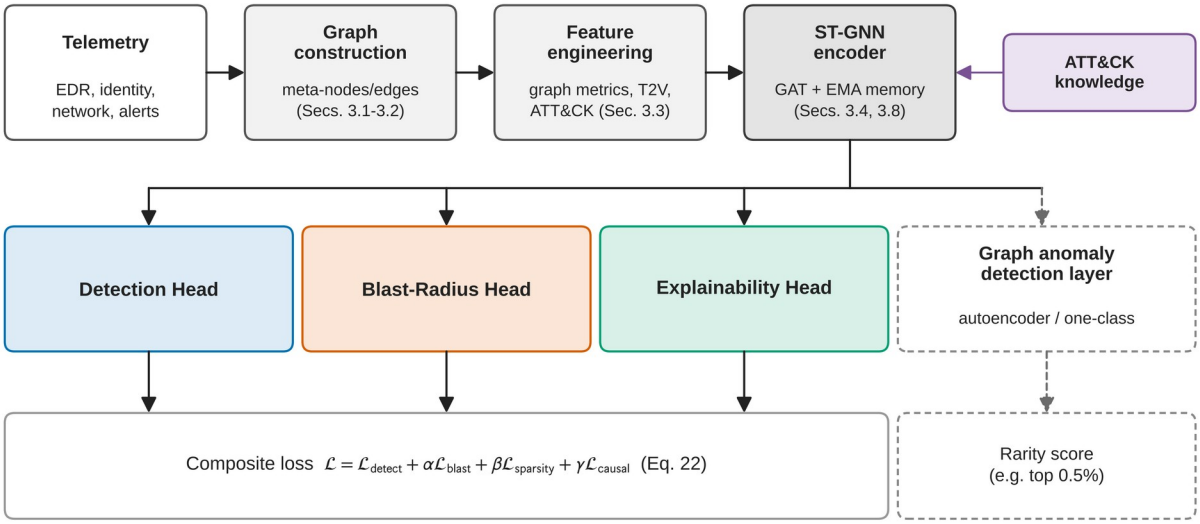


Fig. 7. *Architecture of the proposed Graph-Based Anomaly Detection System (planned extension, not evaluated in this paper).*

3.10 Technical Implementation and Workflow

The experimental framework is engineered as a modular system in which each pipeline stage is encapsulated within a dedicated module, separating data engineering, feature enrichment, and model orchestration.

3.10.1 Data Ingestion and Temporal Processing (`data/`). `preprocess.py` ingests the raw, IP-address-inclusive UNSW-NB15 CSV files, performs data sanitisation (malformed port encodings, infinite and missing values), and maps attack categories to a task-specific label taxonomy via an injectable label-map mechanism, retaining the seven attack categories with sufficient support and excluding three near-empty categories by an explicit, logged threshold (Section 5.1); excluded rows are dropped, not relabelled as benign. A per-capture-session, attack/benign-stratified temporal split is enforced so the model is trained on past data and evaluated on subsequent events, while guaranteeing every retained attack category is represented in train, validation, and test [14].

`graph_builder.py` constructs the Dynamic Threat Graph, mapping unique IP addresses to nodes and aggregating flows sharing a (source IP, destination IP, time-window) tuple into meta-edges. Both the graph-level snapshot label and each meta-edge's label are assigned by **plurality vote** over the label distribution within the corresponding window, rather than by the highest-index attack class present – the correction formalised as Proposition 1 in Section 4.2. The module additionally looks ahead to subsequent time bins to generate ground-truth labels for the Blast-Radius Head, identifying which nodes were compromised in later stages of the attack; the coverage and adequacy of this look-ahead labelling on UNSW-NB15 specifically is addressed in Section 4.1 and Section 5.4.

3.10.2 Multi-Dimensional Feature Engineering (`features/`). `node_features.py` computes graph-theoretic metrics for each node, including Eigenvector Centrality, Temporal Burstiness, and Proximity to Crown Jewels (multi-source BFS). `time2vec.py` and `mitre.py` enrich edges with learnable temporal encodings (Time2Vec) and semantic embeddings; the MITRE embedder uses a frozen

Sentence-Transformer (BERT) to project MITRE ATT&CK technique descriptions into a 64-dimensional latent space, allowing the GNN to learn the semantic content behind specific adversarial behaviours.

3.10.3 Multi-Task Model Architecture (`models/`). The core computation occurs within the three-head spatiotemporal architecture. `st_gnn.py` implements the `STGNNEngine`, using GAT layers with a Temporal Carry-over mechanism that maintains state across sequential 5-minute snapshots. `heads.py` implements the three core task-specific heads, plus the two optional rare-class-recovery heads (Section 3.6.1): the Detection Head performs global attention pooling for graph-level attack classification; the Blast-Radius Head executes node-level regression to forecast infection probabilities; the Explainability Head learns an edge-masking function to extract the most influential causal subgraph.

`trainer.py` orchestrates the training loop and manages the Node Memory module described in Section 3.8. `loss.py` computes the composite multi-task loss (Eq. 22), balancing classification accuracy, propagation-forecasting error, explanation sparsity, and causal alignment with MITRE techniques.

3.10.4 Evaluation and Operational Insights (`evaluation/`, `results/`). `metrics.py` implements standard F1-scores together with specialised metrics – NDCG@K and Hits@K for ranking risk, and Sparsity Ratio / MITRE-Consistency for explanation quality. `visualizer.py` is designed to transform raw model outputs into an operational dashboard, providing an ASCII-based attack narrative, ranked IP risk tables, and recommended mitigation actions for security analysts; this dashboard is a design target of the pipeline and is not itself an evaluated contribution of this paper.

4 Formal Properties of the Framework

This section develops the theoretical content that a framework paper without comparative baselines must supply in its place: a precise statement of the learning problem each head solves, a proof-level treatment of the label-aggregation result that motivates the rare-class-recovery mechanisms of Section 3.6.1, a complexity analysis of the full pipeline, and the design rationale behind the three-head, shared-loss architecture.

4.1 Blast-Radius Forecasting as a Supervised Learning Problem

Section 3.6.2 introduced the Blast-Radius Head informally, as a component that estimates node-level compromise probability. This subsection states the learning problem it is intended to solve as a formal supervised-learning specification, separately from any dataset's ability to supply labels for it. Doing so makes explicit exactly which assumption UNSW-NB15 fails to satisfy, which is the reason the head is implemented and jointly trained but not evaluated in this paper (Section 5.4).

Input space. At time t, the input is the pair (G_t, i), where $G_t = (V_t, E_t, \mathbf{X}_t)$ is the dynamic threat graph defined in Eq. (3) and $i \in V_t$ is a node. The representation actually consumed by the head is the pair of embeddings $(h_i^{(L)}, z_{G_t})$ produced by the shared encoder (Eq. 16), so the input space as seen by the head is $\mathscr{H} \times \mathscr{Z} \subseteq \mathbb{R}^d \times \mathbb{R}^d$ for encoder width d.

Output space. The head produces $\hat{p}_i \in [0,1]$ (Eq. 17), an estimate of $P(\text{compromised_soon} \mid G_t, i)$.

Label-generating process. For a future horizon Δ, define

$$y_i^{\text{future}} = \mathbf{1}\Big[\, i \text{ is compromised for the first time within } (t, t+\Delta] \,\Big]$$

The qualifier *for the first time* is load-bearing and is the source of the assumption below: the target is *new* compromise, conditioned on node i not already being compromised at time t. A model trained against a label that instead marks *any* node showing continued attack activity – including nodes already compromised at t that remain active – is not being trained on propagation; it is being trained to re-detect an already-detected state, which is a different and much easier problem that would trivially inflate any ranking metric computed against it.

Objective functional. The population-level objective is

$$\min_{\theta} \; \mathbb{E}_{(G_t, i)\sim\mathcal{D}}\Big[\, \mathrm{CE}\big(y_i^{\text{future}}(i),\; \hat{p}_i^{\theta}(G_t)\big)\Big]$$

with the empirical form given in Eq. (18), θ the shared-encoder-plus-head parameters, and $\mathcal{D}$ the distribution over (snapshot, node) pairs induced by the training split.

Assumptions. The formulation above is sound only if the following hold of the data-generating process and the dataset used to instantiate it:

- **(A1) Observability of first compromise.** The dataset must record, for a non-trivial number of nodes, the *time of first compromise*, distinguishable from continued activity by an already-compromised node. Without this, y_i^{future} as defined cannot be constructed, and any proxy label constructed in its place is measuring a different quantity than the one stated above.
- **(A2) A meaningful base rate of new compromise events within the evaluation window.** If the number of true first-compromise events in the test split is a small fraction of the positive class, ranking metrics computed against it (Hits@K, NDCG@K, AUC-ROC) have high variance and low statistical power, independent of model quality.
- **(A3) Node identity is not a shortcut to the label.** If a small, closed set of nodes accounts for most or all positive labels across the dataset's timeline, a model can achieve non-trivial ranking performance by memorising which node IDs tend to be positive, rather than by learning behavioural propagation signal. This is the concern the robustness check in Section 5.3 targets for the Detection Head; the same concern applies, with greater force, to the Blast-Radius Head, because propagation labels are typically sparser and more node-concentrated than detection labels.
- **(A4) The look-ahead horizon Δ is chosen independently of the evaluation split boundary.** A horizon that extends past the end of the available capture window systematically censors true positives near the split boundary, biasing the empirical label distribution toward negatives.
- **(A5) Propagation is graph-local at the timescale of $\Delta\tau$.** The message-passing depth L (Section 3.4) bounds the radius over which information reaches $h_i^{(L)}$; if true propagation frequently occurs between nodes more than L hops apart within a single snapshot, the representation available to the head at Eq. (16) is by construction insufficient, independent of training data volume.

Why UNSW-NB15 does not satisfy this specification. UNSW-NB15's raw export records flow-level attack labels, not host compromise events, and its underlying testbed contains 48 IP addresses. Section 5.4

reports the specific consequence measured against this dataset – a positive-test-case count for genuinely new compromise so small that (A2) fails outright – and states this as the reason the head, while implemented and jointly trained throughout the feasibility study, is not evaluated. This formalisation is what makes that omission a *dataset limitation identified in advance of reporting a result*, rather than an unexplained absence: (A1)–(A2) are properties of the label supply, checkable before any model is trained, and Section 6 specifies the dataset properties an evaluation study must establish before reporting Blast-Radius metrics.

4.2 Label Aggregation for Window-Based Dynamic-Graph Classification

This subsection states, as a proposition with a short proof, the structural claim that motivates the two rare-class-recovery mechanisms of Section 3.6.1, and distinguishes it from a second, independent labelling error identified during development.

Setup. Fix a time window w containing a multiset of flow-level attack labels $\{c_1, \ldots, c_{nw}\}$ drawn from a taxonomy $\mathscr{C} = \{c^{(1)}, \ldots, c^{(K)}\}$ indexed arbitrarily 1, …, K. A **single-label window-aggregation rule** is a function $A: \mathscr{C}^{nw} \to \mathscr{C}$ that assigns the window one label. Two rules are of interest:

- **Plurality rule** $A_{plur}(w) = \text{argmax}_{c \in \mathscr{C}} |\{k : c_k = c\}|$ (ties broken arbitrarily), assigning the window its most frequent flow-level label.
- **Max-index rule** $A_{idx}(w) = c^{(\max\{k : c_k \text{ present in } w\})}$, assigning the window the label with the highest taxonomy index among those present, irrespective of frequency.

Proposition 1 (Structural invisibility under single-label window aggregation). *Let $\mathscr{W}$ be the set of all windows in a dataset, and let $c \in \mathscr{C}$ be an attack class such that c is never the plurality label in any window – i.e., $\forall w \in \mathscr{W}, A_{plur}(w) \neq c$ – despite c appearing as a flow-level label with positive frequency in one or more windows. Then no single-label, window-level classifier trained on labels $\{A_{plur}(w)\}_{w \in \mathscr{W}}$ can achieve nonzero recall on class c at the window level, regardless of model capacity, architecture, or optimisation procedure.*

Proof. A window-level classifier trained on single-label targets is, by construction, a function $f: \mathscr{G}_w \to \mathscr{C}$ optimised against the empirical distribution of $\{A_{plur}(w)\}$. Because c is never the value of $A_{plur}(w)$ for any w in the training data by hypothesis, the training label $y_w = c$ never occurs; the empirical loss (e.g., Eq. 15) places zero weight on $\hat{y} = c$ being correct at training time, and standard supervised-learning generalisation guarantees only extend to the support of the training label distribution. At test time, ground truth constructed by the same rule A_{plur} likewise never assigns label c to a window, so recall on class c – defined as $P(\hat{y}_w = c \mid y_w = c)$ – has an undefined or vacuously zero numerator: there are no windows with $y_w = c$ against which recall could be computed as positive. A model could in principle predict c for some window and be scored as a false positive against every such prediction, but it can never be scored as a true positive, so recall (and hence F1) on class c is identically zero for any classifier trained and evaluated under A_{plur}-generated single-label window targets. ∎

The proposition is stated for A_{plur} because plurality is the *best-case* single-label rule: it is the aggregation choice that discards the least information. The max-index rule A_{idx} used in an earlier version of the pipeline is strictly worse in a precise sense: it does not merely inherit the structural blind spot described in Proposition 1 (any class not present as the highest-indexed class in some window is equally invisible under

A_{idx}), it additionally **mislabels windows non-randomly** whenever two or more attack types co-occur – every such window is assigned the higher-indexed class regardless of which class is actually dominant in flow volume, systematically transferring training signal from lower-indexed classes to higher-indexed ones. This second effect has no analogue under A_{plur} and is a labelling *error*, not merely a labelling *ceiling*; identifying and correcting it (switching $A_{idx} \rightarrow A_{plur}$ throughout `graph_builder.py`, Section 3.10.1) is reported as a methodological finding of this work because it is a general failure mode of window-based dynamic-graph labelling on traffic with concurrent attack types, not specific to UNSW-NB15 or to this architecture.

Corollary (why plurality alone is not sufficient). Proposition 1 applies unchanged to A_{plur}: correcting max-index to plurality removes the systematic mislabelling but does not remove the structural ceiling – any class that is a real but consistently minority presence within every window it appears in remains permanently unrecoverable at the window level under *any* single-label rule. This is precisely the gap the two mechanisms of Section 3.6.1 are designed to close: multi-label detection removes the single-label constraint at the window level by allowing more than one class to be positive per window (subject to the presence threshold in Section 3.6.1), and per-edge classification removes window-level pooling entirely, supervising each class directly in proportion to its flow-level frequency. Section 5.2 reports the empirical magnitude of the effect on UNSW-NB15: classes at window-level F1 = 0 under the uncorrected rule reach edge-level F1 ≥ 0.94 once both mechanisms are in place.

4.3 Computational Complexity

Let n be the number of meta-nodes and m the number of meta-edges in a snapshot G_t after the aggregation of Section 3.1, d the hidden dimension, L the number of GAT layers, H the number of attention heads per layer, and K the number of detection classes. Table 5 summarises per-snapshot cost; Appendix A gives the derivation for each row.

Stage	Time complexity	Space complexity
Entity aggregation (§3.1)	$O(n_{raw})$	$O(n)$
Node feature computation, incl. centrality power-iteration step and BFS-based crown-jewel distance (§3.3)	$O(m + n \log n)$ per update	$O(n + m)$
MITRE semantic edge embedding (§3.3, §3.5), amortised	$O(m \cdot d)$	$O(\lvert\mathscr{C}_{mitre}\rvert \cdot d)$
One GAT layer, H heads (§3.4)	$O(H \cdot m \cdot d)$	$O(H \cdot (n+m) \cdot d)$
Full encoder, L layers	$O(L \cdot H \cdot m \cdot d)$	$O(H \cdot (n+m) \cdot d)$
Detection Head, single-label or multi-label (§3.6.1)	$O(n \cdot d + K \cdot d)$	$O(K \cdot d)$
Per-edge classification head (§3.6.1)	$O(m \cdot d + K \cdot d)$	$O(K \cdot d)$
Blast-Radius Head (§3.6.2)	$O(n \cdot d)$	$O(d)$
Explainability Head, mask + sparsity (§3.6.3)	$O(m \cdot d)$	$O(m)$
Composite loss assembly (Eq. 22)	$O(n + m)$	$O(1)$
Node-memory EMA update (§3.8)	$O(n \cdot d)$ per snapshot	$O(n \cdot d)$ persistent

Table 5. Per-snapshot computational and memory complexity of the framework's pipeline stages and heads.

Two properties are worth stating explicitly because they determine how the framework scales.

Linearity in m. Every stage that dominates total runtime – the GAT encoder and the two edge-supervised heads – is linear in the number of meta-edges m, not quadratic in the number of meta-nodes n. This is a direct consequence of using attention restricted to the observed edge set E_t (Eq. 11) rather than dense pairwise attention over V_t; it is the property that makes the encoder's cost tied to the sparsity of observed communication rather than to the square of the node count, which is the relevant scaling regime for enterprise networks where m grows with observed traffic but n is bounded by host and identity inventory.

Node memory as the sequential bottleneck. The EMA node-memory update (§3.8) is the only stage in Table 5 with a hard temporal dependency: snapshot t's memory update reads the state left by snapshot t-1, so encoder forward passes cannot be parallelised across snapshots without either relaxing the recurrence or checkpointing memory state at intervals. Every other row is embarrassingly parallel across snapshots within an epoch. This identifies precisely where a production deployment's throughput ceiling would come from, and is stated here as an architectural property rather than a measured throughput number, which Section 6 lists among the experiments the evaluation study should report.

4.4 Design Rationale

Why three heads over a shared encoder, rather than three separate models. A shared encoder forces the representation $h_i^{(L)}$ to be simultaneously useful for classifying the current window, ranking future compromise risk, and supporting a sparse, MITRE-aligned edge mask. This is a deliberate multi-task constraint in the sense of [14]: it is intended to regularise the encoder against overfitting to whichever single objective would otherwise dominate, at the acknowledged risk of negative transfer between the three heads if their gradients conflict. Because α, β, γ in Eq. (22) are free hyperparameters, the practical trade-off between positive multi-task regularisation and negative transfer is an empirical question, not a claim this paper resolves; it is listed explicitly as an ablation the evaluation protocol (Section 6.2) is designed to answer, rather than argued for on the training data used here.

Why the composite loss is additive rather than a learned combination. A fixed-weight additive loss (Eq. 22) is the simplest instantiation of the multi-task objective and is what the feasibility study in Section 5 trains against. Learned or uncertainty-weighted combinations of the four terms are a documented alternative in the multi-task learning literature and are named in Section 6.2 as part of the proposed ablation matrix rather than adopted here, so that the feasibility study reports results under the simplest, most auditable version of the objective.

Why detection, propagation, and explanation are coupled rather than sequential (detect → explain → forecast). A sequential pipeline in which detection output feeds a separate explanation model, which in turn feeds a separate forecasting model, compounds errors across stages and gives the explanation and forecasting components no gradient signal from the objectives they are meant to support. Coupling all three through a shared encoder means the representation that produces the detection decision is the same representation the Explainability Head must justify and the Blast-Radius Head must extrapolate from, which is the architectural claim RQ1–RQ3 (Section 1.5) are about; Section 6 is where that claim is proposed to be tested against a sequential/decoupled ablation.

5 Feasibility Study on UNSW-NB15

This section reports a feasibility study of the framework specified in Sections 3–4: an instantiation of the full architecture, trained end-to-end on real, IP-address-labelled traffic, intended to demonstrate that the pipeline runs, trains stably, and produces a learnable signal – not to establish comparative performance. Every number below comes from the run configuration described in this section; no figure has been estimated, interpolated, or carried over from a different dataset or task formulation.

5.1 Dataset and Experimental Setup

The feasibility study uses UNSW-NB15 [25], produced by the Australian Centre for Cyber Security using the IXIA PerfectStorm tool to generate a hybrid mixture of real, contemporary benign traffic and synthetic attack activity across nine attack categories. Unlike the dataset's pre-partitioned training/testing CSVs, which omit source/destination IP addresses and cannot support graph construction, this study uses the original, IP-address-inclusive raw export (`UNSW-NB15_1-4.csv`): 2,540,047 flow records, of which 321,283 (12.6%) carry a real attack label.

Real attack traffic in the raw export is concentrated in two capture sessions – 22–23 January 2015 and 18 February 2015 – rather than spread across the file's nominal date range; these two sessions are treated as independent "days" for the temporal split, analogous to per-day splitting on multi-day enterprise captures. Three of the nine attack categories fall below an explicit, logged support threshold of 50 real training edges – Backdoor (0), Analysis (15), Worms (2) – jointly 1.6% of all real attack traffic, and are excluded from train, validation, and test alike (dropped, not relabelled as benign), leaving a seven-class taxonomy: Benign, DoS, Generic, Exploits, Fuzzers, Reconnaissance, Shellcode.

Meta-edges are constructed as described in Section 3.10.1, using a five-minute aggregation window. A per-session, attack/benign-stratified temporal split allocates 70% of snapshots to training, 15% to validation, and 15% to test, computed separately within attack-containing and pure-benign time bins before pooling, to guarantee every retained attack category is represented in all three splits and to avoid the degenerate case where an entire attack episode falls into a single split under a naive chronological cut. This yields 202 training, 44 validation, and 46 test graph-level snapshots, and approximately 4,700–4,900 meta-edges per split.

The instantiated encoder uses 3 GAT layers with 4 attention heads and hidden dimension 128, with the EMA node-memory mechanism of Section 3.8 active throughout training. All results are reported as mean ± standard deviation over five random seeds (42–46), with identical architecture, data split, and hyperparameters across seeds except where noted.

5.2 Detection Results

Table 1 reports overall detection performance at the two levels of supervision described in Section 3.6.1: window level, using the multi-label Detection Head, and meta-edge level, using the per-edge classification head. Table 2 reports the per-class breakdown at both levels.

Metric	Score (mean ± s.d., 5 seeds)
Edge-level Macro F1	0.978 ± 0.018
Graph-level Macro F1 (multi-label)	0.849 ± 0.066
Graph-level macro Average Precision	0.839 ± 0.076

Table 1. Overall detection metrics on the seven-class UNSW-NB15 taxonomy.

Class	Graph-level F1 (multi-label)	Edge-level F1
Benign	0.842 ± 0.116	1.000 ± 0.000
DoS	0.853 ± 0.085	0.963 ± 0.050
Generic	0.878 ± 0.055	0.941 ± 0.057
Exploits	0.916 ± 0.064	0.999 ± 0.001
Fuzzers	0.901 ± 0.054	1.000 ± 0.000
Reconnaissance	0.916 ± 0.064	0.999 ± 0.001
Shellcode	0.641 ± 0.081	0.943 ± 0.070
Macro	**0.849 ± 0.066**	**0.978 ± 0.018**

Table 2. Per-class F1, graph level (multi-label Detection Head) vs. edge level (per-edge classification head).

Every retained class reaches edge-level F1 ≥ 0.94. This is the central empirical result of the feasibility study, and it is a direct empirical instance of the mechanism formalised in Section 4.2: under the corrected, plurality-based single-label window rule, Reconnaissance, Shellcode, and DoS are precisely the categories that Proposition 1 predicts would remain invisible at the window level whenever they are a real but non-dominant presence in the windows where they occur; the per-edge classification head, which is not subject to window-level pooling, recovers each of them to F1 ≥ 0.94 by supervising the same underlying flows directly. Figure 8 presents this window-level-versus-edge-level contrast per class.



Fig. 8. *Per-class F1 at window level (multi-label Detection Head) versus meta-edge level (per-edge classification head), mean ± s.d. over 5 seeds. The gap between the two bars for each class is the empirical signature of the structural ceiling formalised*

in Proposition 1 (Section 4.2): classes with the largest gap are those most often present as a real but non-plurality share of their windows.

5.3 Identity-Robustness Check

Section 4.1 (A3) identifies node-identity memorisation as a concern for any head that conditions on per-node representations, and notes that a full IP-disjoint train/test split is the direct way to rule it out. UNSW-NB15's testbed contains only 48 unique IP addresses in total – too few to hold any out and still retain a usable split – so a full IP-disjoint test is not possible on this dataset. As a targeted proxy, the one node feature computed against a role assignment fixed across the entire dataset rather than recomputed per window – distance to crown jewels, Eq. (8) – is masked and the model retrained and re-evaluated under otherwise identical settings.

Configuration	Graph-level Macro F1	Edge-level Macro F1
Full feature set (Table 1)	0.849 ± 0.066	0.978 ± 0.018
Distance-to-crown-jewels masked	0.855 ± 0.084	0.990 ± 0.006

Table 3. Identity-robustness check: masking the one fixed, dataset-wide node-role feature leaves performance unchanged (graph level) or slightly improved and more stable (edge level).

Performance is unchanged at the graph level and slightly improved and more stable at the edge level with this feature removed, which is evidence against the reported result being driven by memorisation of this particular fixed per-node signal. This check is narrower than a true IP-disjoint generalisation test – which this dataset's 48-IP testbed cannot support – and Section 6.1 lists a dataset with a larger, disjoint IP population among the requirements for a stronger version of this test.

5.4 Blast-Radius Head: Implementation Status

The Blast-Radius Head (Section 3.6.2) is implemented and jointly trained, but is not evaluated here: UNSW-NB15 contains only 2 of 313 positive test cases representing genuinely new compromise rather than persistence of existing attack activity, which is insufficient to assess propagation forecasting. Its evaluation is left to future work on datasets with richer propagation ground truth. This is precisely the assumption (A2) of Section 4.1 failing to hold on this dataset: with a test-set base rate this low, no ranking metric computed against it – Hits@K, NDCG@K, or AUC-ROC – would carry statistical power sufficient to distinguish genuine propagation signal from noise, independent of the head's underlying quality. Section 6.1 specifies the dataset property an evaluation study must verify – a non-trivial count of labelled first-compromise events – before reporting Blast-Radius metrics as a result rather than as a diagnostic.

5.5 Explainability: Sparsity, and a Known Limitation in MITRE-Consistency

The Explainability Head (Section 3.6.3) produces an Explanation Sparsity Ratio of 0.993 ± 0.015, i.e. under 1% of edges are flagged as explanatory on average, consistent with the sparsity objective in Eq. (21) exerting a strong, stable pull toward compact explanation subgraphs across seeds. Table 4 reports the two explainability metrics together for completeness.

Metric	Score (mean ± s.d., 5 seeds)
Explanation Sparsity Ratio	0.993 ± 0.015
MITRE-Consistency	Not reported – see limitation below

Table 4. Explainability metrics.

A high sparsity ratio alone does not establish that the selected edges are the *causally correct* minimal subgraph rather than an under-constrained head converging toward a trivial low-sparsity-loss solution; MITRE-Consistency (Eq. 9) is the metric intended to make that distinction. While preparing these results, we identified that the current implementation of the MITRE-Consistency computation references a fixed, CSE-CIC-IDS2018-specific class-to-MITRE-technique mapping rather than the UNSW-NB15-specific mapping used elsewhere in the pipeline (label mapping, class weighting, class names). Because the number this would produce is known to be computed against the wrong technique mapping, it is not reported here rather than silently included; it is recorded as a known implementation defect with a planned fix in the Limitations (Section 7) and is a precondition, not merely a nice-to-have, for the explainability evaluation proposed in Section 6.3.

5.6 Scope Note on Comparative Evaluation

A comprehensive comparative evaluation against alternative architectures is beyond the scope of this paper, which establishes the framework and its formal properties; such an evaluation is the subject of ongoing work. Section 2.2 names the closest published work on this dataset family – self-supervised edge-embedding detection [26] and timestamp-aware spatiotemporal contrastive detection [27] – as the intended comparison targets, and Section 6 specifies the baselines, ablations, and metrics an evaluation study would need to report against them; none of those numbers are reproduced or estimated here.

6 Proposed Evaluation Protocol

This section specifies the experiments an evaluation study should run to substantiate the framework's performance claims. It is written as a protocol to be executed, not as a set of results; every item below is a design specification with no numbers attached, consistent with the truth constraints stated in Section 1.2. Appendix C tabulates baselines, ablations, and metrics together for reference.

6.1 Comparative Baselines

An evaluation study should report the framework against, at minimum: (i) a single-task detection-only ST-GNN using the same encoder (Section 3.4) and only L_{detect}, isolating what the multi-task objective adds or costs relative to Section 4.4's design rationale; (ii) a non-graph sequence baseline (e.g., an LSTM or Transformer over per-node or per-flow feature sequences) to establish whether the graph structure itself, rather than the temporal modelling alone, contributes to detection quality; (iii) the closest published GNN-based approaches on this dataset family – self-supervised edge-embedding detection [26] and timestamp-aware spatiotemporal contrastive detection [27] – reproduced under the seven-class, IP-address-inclusive task formulation used here rather than cited at face value from their original, differently posed tasks (Section 2.2), since the numbers reported in those papers are not directly

comparable to a differently posed task; and (iv) for the Blast-Radius Head specifically, before any ranking metric is reported, verification that the target dataset satisfies assumptions (A1)–(A2) of Section 4.1 – a non-trivial, labelled count of first-compromise events in the test split – since Section 5.4 shows this framework's own feasibility-study dataset fails that precondition.

6.2 Ablation Study

The design rationale of Section 4.4 identifies three architectural choices that are asserted but not tested in this paper; each should be the subject of one ablation arm. (i) Shared vs. separate encoders per head, to measure the direction and magnitude of transfer between the detection, blast-radius, and explainability objectives. (ii) Fixed-weight additive loss (Eq. 22) vs. a learned or uncertainty-weighted combination of L_{detect}, L_{blast}, $L_{sparsity}$, L_{causal}, to establish whether the simplest version of the objective used in Section 5 leaves performance on the table. (iii) Coupled (shared-encoder) vs. sequential (detect → explain → forecast, with gradients blocked between stages) architectures, to test directly whether coupling the three heads produces the representational benefit claimed qualitatively in Section 4.4. A fourth arm should isolate the two rare-class-recovery mechanisms of Section 3.6.1 from each other – multi-label detection alone, per-edge classification alone, and both together – to attribute the recovery shown in Section 5.2 and Figure 8 to each mechanism individually rather than to their combination only.

6.3 Explainability Validation

Before MITRE-Consistency can be reported as a result rather than withheld as in Section 5.5, the class-to-MITRE-technique mapping used in the loss term (Eq. 9) and the evaluation metric must be verified as UNSW-NB15-specific, matching the mapping already used elsewhere in the pipeline (label mapping, class weighting, class names; Section 3.10.1). Once corrected, an evaluation study should report MITRE-Consistency alongside Sparsity Ratio, and should additionally include a human-evaluation component – SOC analysts rating a sample of generated explanation subgraphs for narrative coherence and actionability – since sparsity and MITRE-alignment are necessary but not sufficient proxies for whether an explanation is actually useful to an analyst, a gap this paper does not close.

6.4 Scaling and Throughput

Section 4.3 derives the framework's asymptotic cost analytically; an evaluation study should complement this with measured wall-clock throughput as a function of m (meta-edges per snapshot) and of the node-memory checkpointing interval identified as the sequential bottleneck in Section 4.3, to establish the practical, not just asymptotic, cost of production-scale deployment.

6.5 Falsification Criteria

Consistent with treating this as a protocol rather than a foregone conclusion, an evaluation study should state in advance the outcomes that would count against the framework's central claims: multi-task coupling failing to outperform the single-task and sequential ablations of Section 6.2 would undercut the design rationale of Section 4.4; the per-edge and multi-label mechanisms failing to reproduce the recovery pattern of Section 5.2 on a second dataset would limit the label-aggregation result of Section 4.2 to a UNSW-NB15-specific artefact rather than a general one; and a Blast-Radius Head that, once evaluated on

a dataset satisfying (A1)–(A2), fails to exceed a simple recency or degree-based propagation heuristic would indicate the head's added value is not established even where its evaluation is finally possible.

7 Limitations

7.1 Bounded attack taxonomy. Three of UNSW-NB15's nine attack categories (Backdoor, Analysis, Worms) are excluded from the reported results by an explicit support threshold (fewer than 50 real training edges each – 0, 15, and 2 respectively), 1.6% of all real UNSW-NB15 attack traffic. This is a genuine data-scarcity limit rather than an artefact of the modelling approach – no window-labelling or class-balancing scheme recovers signal from that few real examples – but the reported taxonomy is narrower than the dataset's full attack ontology, and detection of these specific attack types is untested.

7.2 Small, bounded evaluation set. UNSW-NB15's raw export concentrates real traffic into two capture sessions, yielding 44–46 graph-level snapshots per split; the per-edge classification head expands this to roughly 4,700–4,900 evaluation examples, but both remain bounded by the same two real captures rather than an open-ended traffic stream. Results are reported as mean ± s.d. over five random seeds throughout for this reason and should be read with that variance in mind rather than as single point estimates.

7.3 No IP-disjoint generalisation test. The most direct check for whether a graph-based model generalises to previously unseen infrastructure, rather than partially exploiting node identity, is a train/test split with disjoint IP sets. UNSW-NB15's synthetic testbed contains only 48 unique IP addresses in total, too few to support such a split. Section 5.3 instead reports a targeted feature-ablation robustness check, which showed no performance degradation; this is evidence against pure identity memorisation of the one fixed feature tested but is narrower than a true IP-disjoint generalisation test, which this dataset cannot support.

7.4 MITRE-Consistency not yet dataset-aware. The MITRE-Consistency metric and the causal-consistency loss term (Eq. 9) currently reference a fixed, CSE-CIC-IDS2018-specific class-to-MITRE-technique mapping rather than the dataset-specific mapping already used elsewhere in the pipeline. This was identified while preparing the results in Section 5.5, and the metric is withheld rather than reported against the wrong mapping; a fix is planned and is a precondition for the explainability evaluation proposed in Section 6.3.

7.5 Blast-Radius Head not evaluated. As stated in Section 5.4, UNSW-NB15's test split contains only 2 of 313 positive cases representing genuinely new compromise, which is insufficient statistical support for any ranking metric. The head is implemented and jointly trained throughout this study, but no performance number for it is reported anywhere in this paper.

7.6 No comparative baselines or ablations in this paper. Consistent with the paper's scope as stated in Section 1.2, no baseline architecture and no ablation of the design choices in Section 4.4 is reported here; Section 6 specifies what such a study should contain, and it is reserved for a dedicated evaluation paper.

7.7 Single-dataset evaluation. All results in this paper are on UNSW-NB15. Extending this pipeline to other public benchmarks – including CSE-CIC-IDS2018 [16], several of whose redistributed per-day files omit the IP-address fields required for graph construction, a data-availability constraint independent of this architecture – remains important future validation before claiming cross-dataset generality.

7.8 Anomaly-detection layer not implemented. The graph-level anomaly-detection layer described in Section 3.9 is a planned architectural extension. No implementation of it was trained or evaluated as part of this work.

8 Future Research

8.1 Self-supervised pre-training for threat graphs. Future work could explore graph contrastive learning or graph autoencoders to pre-train the ST-GNN engine on large volumes of unlabelled telemetry. This could enhance the model's ability to identify rare or novel campaigns (zero-day patterns) before fine-tuning on specific detection tasks [10]; the anomaly-detection layer of Section 3.9 would be a natural integration point for this direction.

8.2 Graph-RAG integration. Incorporating retrieval-augmented generation at the graph level would allow the model to retrieve similar historical attack subgraphs from external knowledge bases. This would be expected to enrich the Explainability Head with historical context and improve Blast-Radius accuracy based on past precedents, once that head can be evaluated on a dataset satisfying the assumptions of Section 4.1.

8.3 Human-in-the-loop interactive XAI. Developing mechanisms for analysts to provide real-time feedback on the sparse causal subgraphs, which could be used to dynamically calibrate the sparsity and causal-consistency weights (α, β, γ in Eq. 22), aligning the model's outputs with specific SOC workflows and analyst preferences. This connects directly to the human-evaluation component proposed in Section 6.3.

8.4 Adversarial robustness and causal mimicry. Investigating the resilience of the ST-GNN engine against adversarial attacks designed to deceive the detection or explainability heads (e.g., by mimicking benign causal patterns). Developing causally robust training techniques would be relevant for maintaining analyst trust in adversarial settings.

8.5 Multi-domain, cross-tenant TDR. Extending the architecture to handle hybrid threat graphs that span multiple domains, such as correlating endpoint telemetry with SaaS/cloud logs and external identity providers. This would be a step toward a comprehensive, end-to-end blast-radius map across a full digital estate – but only once the Blast-Radius Head itself has been evaluated on a dataset with adequate propagation ground truth (Section 6.1).

9 Conclusion

This paper presented a multi-task Spatiotemporal Graph Neural Network framework for threat detection, blast-radius forecasting, and explainable alert generation, and delivered it as a framework contribution: a full architectural specification, a formalisation of the learning problem each head solves – including an explicit statement of the assumptions blast-radius forecasting requires of its training data – a complexity analysis of every pipeline stage, a general methodological result on label aggregation for window-based dynamic-graph classification, and a proposed evaluation protocol specifying the experiments required to substantiate the framework's performance claims.

A feasibility study on the raw, IP-address-labelled UNSW-NB15 export showed that the framework trains stably across five random seeds and that the label-aggregation correction identified in Section 4.2, combined with the two rare-class-recovery mechanisms of Section 3.6.1, recovers attack categories that are structurally unrecoverable under single-label window classification – Reconnaissance and Shellcode in particular move from window-level F1 = 0 under the uncorrected rule to edge-level F1 ≥ 0.94 – without loss of performance on well-represented classes. The Blast-Radius Head was implemented and jointly trained throughout but, as stated explicitly in Section 5.4, was not evaluated: UNSW-NB15's test split contains only two genuine first-compromise events, short of what any ranking metric could meaningfully assess. No comparative baselines or ablations are reported; Section 6 specifies what such a study requires.

Collectively, these results support the narrower claim this paper makes: that the proposed architecture is a coherent, formally specified, and computationally tractable way to pose detection, propagation forecasting, and explanation as one learning problem, and that it can be instantiated end-to-end on real, IP-labelled traffic. Whether it outperforms alternative architectures, and whether its Blast-Radius Head delivers useful propagation forecasts on data where that question can actually be asked, are left to the evaluation study the protocol in Section 6 specifies.

Declaration of Generative AI and AI-Assisted Technologies in the Manuscript Preparation Process

During the preparation of this work the author used the following AI-assisted tools: (1) Perplexity AI for literature discovery and research; (2) Google NotebookLM for article organisation and reference management; (3) Claude Code (Anthropic) for simulation development; and (4) Claude (Anthropic) for article structuring and organisation. After using these tools, the author reviewed and edited all content as needed and takes full responsibility for the content of the published article.

Data Availability Statement

The experiments in this paper were conducted using the UNSW-NB15 dataset, produced by the Australian Centre for Cyber Security (ACCS) at the University of New South Wales [25]. The raw, IP-address-inclusive export used in this work is freely accessible at: https://research.unsw.edu.au/projects/unsw-nb15-dataset

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of Competing Interests

The author declares that she has no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] Z. Chen, Z. Tian, J. Cui, Dynamic graph representation learning for cyber attack detection, Comput. Secur. 123 (2023) 102938.

[2] M.A. Ferrag, L. Maglaras, Deep learning for cyber security intrusion detection systems: a survey, Comput. Secur. 101 (2021) 102122.

[3] W.L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, Adv. Neural Inf. Process. Syst. 30 (2017).

[4] W. Hassan, A. Bates, D. Marino, Tactical provenance analysis for endpoint detection and response systems, in: Proc. IEEE Symp. Secur. Priv. (S&P), San Francisco, CA, USA, 2020, pp. 1172–1189.

[5] G. Karatas, O. Demir, O.K. Sahingoz, Increasing the performance of machine learning-based IDSs on an imbalanced dataset, IEEE Access 8 (2020) 32150–32162.

[6] S.M. Kazemi, R. Goel, K. Jain, I. Kobyzev, A. Sethi, P. Forsyth, et al., Time2Vec: learning a vector representation of time, arXiv preprint arXiv:1907.05321 (2019).

[7] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: Proc. Int. Conf. Learn. Represent. (ICLR), Toulon, France, 2017.

[8] C. Li, X. Yuan, X. Li, Spatiotemporal graph neural networks for advanced persistent threat detection, Comput. Secur. 118 (2022) 102720.

[9] T.Y. Lin, P. Goyal, R. Girshick, K. He, P. Dollár, Focal loss for dense object detection, in: Proc. IEEE Int. Conf. Comput. Vis. (ICCV), Venice, Italy, 2017, pp. 2980–2988.

[10] K. Liu, Y. Dou, Y. Zhao, X. Ding, X. Hu, R. Zhang, et al., BOND: benchmarking unsupervised outlier node detection on static attributed graphs, Adv. Neural Inf. Process. Syst. 35 (2022) 27021–27035.

[11] S.M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, V.N. Venkatakrishnan, HOLMES: real-time APT detection through correlation of suspicious information flows, in: Proc. IEEE Symp. Secur. Priv. (S&P), San Francisco, CA, USA, 2019, pp. 1137–1154.

[12] T. Nguyen, V.J. Reddi, Graph-based threat propagation prediction, Comput. Secur. 114 (2022) 102584.

[13] A. Pareja, G. Domeniconi, J. Chen, T. Ma, T. Suzumura, H. Kanezashi, et al., EvolveGCN: evolving graph convolutional networks for dynamic graphs, in: Proc. AAAI Conf. Artif. Intell., New York, NY, USA, 2020, vol. 34, pp. 5363–5370.

[14] S. Ruder, An overview of multi-task learning in deep neural networks, arXiv preprint arXiv:1706.05098 (2017).

[15] W. Samek, K.R. Müller, Explainable artificial intelligence in cybersecurity, Comput. Secur. 105 (2021) 102215.

[16] I. Sharafaldin, A.H. Lashkari, A.A. Ghorbani, Toward generating a new intrusion detection dataset and intrusion traffic characterization, in: Proc. 5th Int. Conf. Inf. Syst. Secur. Priv. (ICISSP), Madeira, Portugal, 2018, pp. 108–116.

[17] H. Singh, V. Sharma, R. Kumar, KillChainGraph: knowledge-graph-driven kill chain modelling for multi-stage attack attribution, arXiv preprint arXiv:2508.18230 (2025).

[18] R. Sommer, V. Paxson, Operational challenges of ML-based security systems, Comput. Secur. 102 (2021) 102154.

[19] B.E. Strom, A. Applebaum, D.P. Miller, K.C. Nickels, A.G. Pennington, C.B. Thomas, MITRE ATT&CK: design and philosophy, Tech. Rep., MITRE Corporation, Bedford, MA, USA, 2020.

[20] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: Proc. Int. Conf. Learn. Represent. (ICLR), Vancouver, BC, Canada, 2018.

[21] S. Wang, A.R. Balef, S. Bhattacharya, I. Gupta, X-IDS: a framework for explainable intrusion detection using SHAP-guided graph learning, Comput. Secur. 122 (2022) 102758.

[22] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, P.S. Yu, A comprehensive study of graph neural networks, IEEE Trans. Neural Netw. Learn. Syst. 32 (2020) 4–24.

[23] H. Yuan, H. Yu, S. Gui, S. Ji, Explainability in graph neural networks: a taxonomic survey, IEEE Trans. Pattern Anal. Mach. Intell. 45 (2022) 5782–5799.

[24] L. Zhao, R. Deng, H. Wang, Unified graph learning framework for detection and explanation of cyber attacks, Comput. Secur. 128 (2024) 103102.

[25] N. Moustafa, J. Slay, UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set), in: Proc. Mil. Commun. Inf. Syst. Conf. (MilCIS), Canberra, Australia, 2015, pp. 1–6.

[26] E. Caville, W.W. Lo, S. Layeghy, M. Portmann, Anomal-E: a self-supervised network intrusion detection system based on graph neural networks, arXiv preprint arXiv:2207.06819 (2022).

[27] J. Dai, G. Wu, J. Li, W. Wang, A. He, X. Xiao, Timestamp-aware spatio-temporal graph contrastive learning for network intrusion detection, arXiv preprint arXiv:2606.17109 (2026).

Appendix A. Complexity Analysis – Derivations

This appendix derives each row of Table 5 (Section 4.3).

A.1 Entity aggregation (§3.1). Each raw event e_i is mapped once through Φ (Eq. 1) into a bucket keyed by (HostID, UserID, window index); with a hash-map implementation this is $O(1)$ amortised per event, giving $O(n_{raw})$ total for n_{raw} raw events, and the output size is bounded by the number of distinct buckets realised, $O(n)$.

A.2 Node features (§3.3). Eigenvector centrality is updated by one power-iteration step per snapshot, touching each edge once: $O(m)$. Crown-jewel distance (Eq. 8) is a multi-source shortest-path computation from the set C of critical assets; on a sparse graph a multi-source BFS runs in $O(n + m)$, and the $n \log n$ term in Table 5 is a conservative bound admitting a priority-queue-based variant if edge weights are later introduced. Burstiness (Eq. 5) is $O(1)$ per node given the maintained EMA, i.e. $O(n)$ total.

A.3 MITRE semantic edge embedding (§3.3, §3.5). Technique embeddings $E(v_i)$ are precomputed once per technique and cached ($O(|\mathscr{C}_{\text{mitre}}| \cdot d)$ space, amortised to $O(1)$ per lookup); assembling $\mathbf{e}_{\text{mitre}}(t)$ into each edge's feature vector (Eq. 6) is $O(d)$ per edge, $O(m \cdot d)$ total.

A.4 GAT layer (§3.4). For each edge $(i,j) \in E_t$ and each of H heads, computing the unnormalised attention score (Eq. 11) is $O(d)$ (a fixed-size inner product after the linear projections, which are themselves $O(n \cdot d^2)$ for the projections and dominated by the $O(m \cdot d)$ attention-score term when $m \gg n$, the regime this framework targets); softmax normalisation per destination node is $O(\deg(i))$, summing to $O(m)$ over the graph. Aggregating messages (Eq. 10) is $O(m \cdot d)$. Total per layer: $O(H \cdot m \cdot d)$; over L layers, $O(L \cdot H \cdot m \cdot d)$.

A.5 Heads (§3.6). The Detection Head's READOUT (Eq. 13) is $O(n \cdot d)$ (one pass over node embeddings), and the classification layer (Eq. 14) is $O(d \cdot K)$. The per-edge head applies a fixed-size classifier to each of m edge representations, $O(m \cdot d + m \cdot K) = O(m \cdot d + K \cdot d)$ after amortising the shared classifier weights. The Blast-Radius Head applies f_{blast} (Eq. 16) once per node, $O(n \cdot d)$. The Explainability Head's mask function $g(\cdot)$ (Eq. 19) is applied once per edge, $O(m \cdot d)$, and the sparsity loss (Eq. 21) is a linear sum over the same m values, $O(m)$.

A.6 Node memory (§3.8). The EMA update touches each of the n active nodes' d-dimensional memory vectors once per snapshot: $O(n \cdot d)$ time, $O(n \cdot d)$ persistent space across the full node population being tracked.

Appendix B. Blast-Radius Forecasting – Extended Formal Notes

This appendix extends Section 4.1 with two points too detailed for the main text.

B.1 Relationship between Δ (forecast horizon) and $\Delta\tau$ (snapshot width). The formalisation in Section 4.1 leaves the relationship between the forecast horizon Δ and the snapshot aggregation width $\Delta\tau$ (Eq. 1) unspecified beyond assumption (A4). In practice Δ should be reported as a multiple of $\Delta\tau$ (e.g., $\Delta = 6\,\Delta\tau$ for a 30-minute horizon under 5-minute snapshots), since the label y_i^{future} is necessarily computed at the resolution of whole snapshots; an evaluation study should report results at more than one Δ to characterise the precision/lead-time trade-off, rather than a single arbitrarily chosen horizon.

B.2 Distinguishing propagation from re-detection at the labelling-code level. Section 4.1's label definition requires distinguishing first compromise from persistence of an already-compromised node. Operationally, this means the look-ahead labelling logic in `graph_builder.py` (Section 3.10.1) must track, per node, the snapshot index of first attack involvement, and define $y_i^{\text{future}}(t) = 1$ only for nodes whose first attack involvement falls strictly within $(t, t+\Delta]$ – nodes already flagged at or before t must be excluded from the positive class at time t regardless of whether they remain active through $(t, t+\Delta]$. This is the specific implementation requirement that assumption (A1) of Section 4.1 imposes, and its absence (rather than mere data scarcity) is an equally plausible failure mode for any dataset an evaluation study considers, not only for UNSW-NB15.

Appendix C. Proposed Evaluation Protocol – Summary Table

Study	Question addressed	Compared against	Metric(s)	Precondition
Baseline: single-task detector	Does multi-task coupling help or hurt detection?	Same encoder, L_{detect} only	Graph/edge Macro F1	–
Baseline: non-graph sequence model	Does graph structure add value beyond temporal modelling?	LSTM/Transformer over flow sequences	Graph/edge Macro F1	–
Baseline: published GNN detectors	How does the framework compare to closest published work?	Reproductions of [26], [27] under this task's 7-class formulation	Macro F1, AP	Reproduction on IP-inclusive UNSW-NB15, not cited at face value
Ablation: encoder sharing	Shared vs. separate encoders per head	Three single-head models	Per-head metric deltas	–
Ablation: loss weighting	Fixed-weight vs. learned/uncertainty-weighted loss	Eq. (22) variants	Graph/edge Macro F1, L_{blast}, $L_{sparsity}$	–
Ablation: coupled vs. sequential	Does joint training beat a detect→explain→forecast pipeline?	Gradient-blocked sequential variant	All head metrics	–
Ablation: rare-class mechanisms	Multi-label alone vs. per-edge alone vs. both	Single-mechanism variants	Per-class F1 (cf. Fig. 8)	–
Blast-Radius evaluation	Does the head forecast propagation better than a heuristic?	Recency/degree-based heuristic	Hits@K, NDCG@K, AUC-ROC	Dataset satisfies (A1)–(A2), §4.1
Explainability validation	Are explanations MITRE-consistent and analyst-useful?	Corrected MITRE mapping; SOC analyst ratings	MITRE-Consistency, Sparsity Ratio, human rating	Dataset-specific MITRE mapping fixed, §7.4
Scaling	What is measured throughput at production scale?	Wall-clock vs. m, memory-checkpoint interval	Snapshots/sec, peak memory	–

Table 6. Proposed evaluation protocol, consolidating Sections 6.1–6.4. No cell in this table is a result reported in this paper.